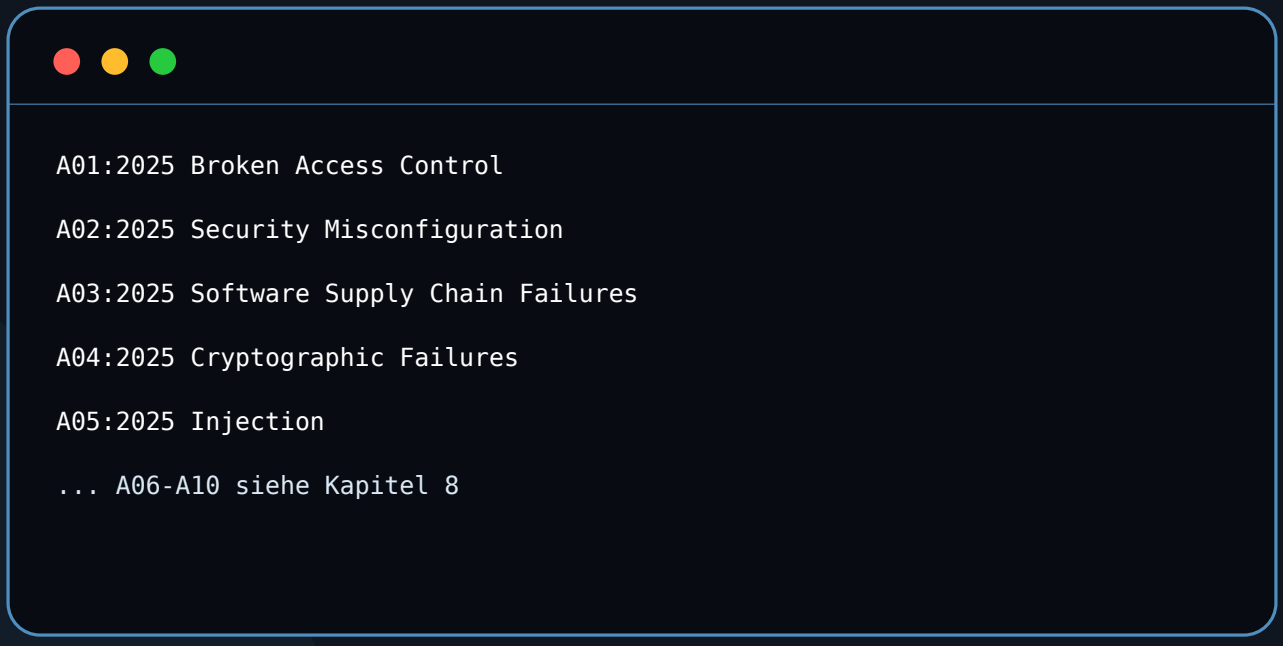
PRAXISHANDBUCH

Web-Security & OWASP

Die OWASP Top 10:2025 verstehen und in der eigenen Praxis anwenden



ZEHN KATEGORIEN. EIN GEMEINSAMES RASTER FÜR DIE GANZE REIHE.

Mit Übungen, Spickzettel, Glossar und Index · Ausgabe Oktober 2026

Web-Security & OWASP

Die OWASP Top 10:2025 verstehen und in der eigenen Praxis anwenden

Von Broken Access Control über Security Misconfiguration bis zur Software Supply Chain. Mit geführten Übungen gegen DVWA, Spickzettel, Glossar und Index.

Inhaltsverzeichnis

Vorwort . . . 5

Für wen ist dieses Handbuch? . . . 5

Aufbau . . . 6

Kapitel 1: Was ist OWASP? . . . 7

1.1 Eine gemeinnützige Stiftung, kein Unternehmen . . . 7

1.2 Die wichtigsten Projekte im Überblick . . . 7

1.3 Warum die Top 10 nicht das ganze Bild sind . . . 7

Zusammenfassung . . . 8

Kapitel 2: Die OWASP Top 10:2025 im Überblick . . . 9

2.1 Die zehn Kategorien . . . 9

2.2 Wie die Reihenfolge entsteht . . . 9

2.3 Was sich seit 2021 verändert hat . . . 10

Zusammenfassung . . . 10

Kapitel 3: A01:2025 – Broken Access Control . . . 11

3.1 Was gehört zu dieser Kategorie? . . . 11

3.2 Warum SSRF jetzt hierher gehört . . . 11

3.3 Mit Burp Suite nachvollziehen . . . 12

3.4 Gegenmaßnahmen . . . 12

Zusammenfassung . . . 12

Kapitel 4: A02:2025 – Security Misconfiguration . . . 13

4.1 Warum diese Kategorie so stark gestiegen ist . . . 13

4.2 Mit Werkzeugen aus dieser Reihe aufspüren . . . 13

4.3 Gegenmaßnahmen . . . 14

Zusammenfassung . . . 14

Kapitel 5: A03:2025 – Software Supply Chain Failures . . . 15

5.1 Mehr als nur veraltete Bibliotheken . . . 15

5.2 Mit OWASP Dependency-Check aufspüren . . . 15

5.3 Gegenmaßnahmen . . . 16

Zusammenfassung . . . 16

Kapitel 6: A04:2025 – Cryptographic Failures . . . 17

6.1 Von Rang 2 (2021) auf Rang 4 (2025) . . . 17
6.2 Querverweise zu anderen Büchern dieser Reihe . . . 17
6.3 Gegenmaßnahmen . . . 18
Zusammenfassung . . . 18

Kapitel 7: A05:2025 – Injection . . . 19

7.1 Von Rang 3 (2021) auf Rang 5 (2025) . . . 19
7.2 Dasselbe Grundprinzip, verschiedene Kontexte . . . 19
7.3 Gegenmaßnahmen . . . 20
Zusammenfassung . . . 20

Kapitel 8: A06–A10:2025 – die übrigen fünf Kategorien . . . 21

8.1 A06:2025 – Insecure Design . . . 21
8.2 A07:2025 – Authentication Failures . . . 21
8.3 A08:2025 – Software and Data Integrity Failures . . . 21
8.4 A09:2025 – Logging and Alerting Failures . . . 22
8.5 A10:2025 – Mishandling of Exceptional Conditions . . . 22
Zusammenfassung . . . 22

Kapitel 9: Von der Kategorie zum Test . . . 23

9.1 Vom Risikobewusstsein zum methodischen Test . . . 23
9.2 ASVS: wie gründlich wurde geprüft? . . . 23
9.3 Die Cheat Sheet Series: konkrete Umsetzung . . . 23
9.4 Die drei Dokumente im Zusammenspiel . . . 24
Zusammenfassung . . . 24

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen . . . 25

10.1 Laboraufbau . . . 25
10.2 Checkliste: sichere Laborumgebung . . . 25
10.3 Übungen . . . 25
Zusammenfassung . . . 27

Kapitel 11: Häufige Missverständnisse, Spickzettel, Glossar und Ressourcen . . . 28

11.1 Häufige Missverständnisse . . . 28
11.2 Spickzettel . . . 28
11.3 Glossar . . . 29
11.4 Weiterführende Ressourcen . . . 29
Zusammenfassung . . . 30

Verzeichnis der Befehlsbeispiele 31

Index 32

Vorwort

Die bisherigen Bände dieser Reihe behandeln einzelne Werkzeuge: hping3 erzeugt Pakete, Nmap erkundet Netzwerke, Burp Suite prüft Webanwendungen. Dieses Buch ist anders aufgebaut. Es behandelt kein einzelnes Werkzeug, sondern eine **gemeinsame Ordnung**: das OWASP-Projekt und seine Top 10, mit der sich die in den anderen Büchern gefundenen Schwachstellen einem festen, weltweit anerkannten Raster zuordnen lassen.

Im Oktober 2026 ist die aktuelle Fassung die **OWASP Top 10:2025**, im November 2025 vorgestellt und im Januar 2026 endgültig veröffentlicht. Sie löst die seit 2021 gültige Fassung ab und liegt damit nach dem Wissensstand ab, auf dem viele ältere Lehrmaterialien noch aufbauen. Dieses Buch arbeitet durchgehend mit der neuen 2025er-Fassung und weist an den Stellen, an denen sich die Reihenfolge oder der Zuschnitt einer Kategorie gegenüber 2021 deutlich verändert hat, ausdrücklich darauf hin.

> **Gut zu wissen**
>
> Dieses Buch stützt sich auf öffentlich recherchierte, aktuelle Angaben zur OWASP Top 10:2025. Prüfe bei wichtigen Entscheidungen zusätzlich die offizielle Quelle unter owasp.org, da sich Einzelheiten nach Redaktionsschluss noch ändern können.

Für wen ist dieses Handbuch?

Du bist hier richtig, wenn du

- die in den anderen Büchern dieser Reihe gefundenen Schwachstellen einer anerkannten Kategorie zuordnen möchtest, statt sie isoliert zu betrachten,
- als angehende Fachkraft für Systemintegration verstehen willst, worüber Auftraggeber und Prüfberichte sprechen, wenn von „OWASP-Konformität" die Rede ist,
- dich auf Zertifizierungen vorbereitest, in denen die OWASP Top 10 ausdrücklich Teil des Prüfungsstoffs sind (eJPT, OSCP, CEH),
- von der 2021er- auf die 2025er-Fassung umsteigen und die Unterschiede konkret verstehen möchtest.

Aufbau

Kapitel	Thema
1	Was ist OWASP? Projekte im Überblick
2	Die OWASP Top 10:2025 im Überblick und der Wandel seit 2021
3	A01:2025 – Broken Access Control
4	A02:2025 – Security Misconfiguration
5	A03:2025 – Software Supply Chain Failures
6	A04:2025 – Cryptographic Failures
7	A05:2025 – Injection
8	A06–A10:2025 – die übrigen fünf Kategorien
9	Von der Kategorie zum Test: ASVS, Testing Guide, Cheat Sheet Series
10	Praxisteil: Laboraufbau und geführte Übungen
11	Häufige Missverständnisse, Spickzettel, Glossar und Ressourcen

> **TIPP – Profi-Trick**
>
> Fast jedes praktische Beispiel in diesem Buch lässt sich mit einem Werkzeug aus einem anderen Band dieser Reihe nachvollziehen: Burp Suite für die HTTP-Ebene, Nmap für die Serverkonfiguration, John the Ripper oder Hashcat für Kryptografie-Versagen. Die Querverweise im Text zeigen jeweils, welches Buch passt.

Kapitel 1: Was ist OWASP?

Lernziele

- Du kennst Zweck und Organisationsform von OWASP.
- Du kannst die wichtigsten OWASP-Projekte benennen und voneinander abgrenzen.
- Du verstehst, warum die Top 10 nur eines von vielen OWASP-Projekten sind.

1.1 Eine gemeinnützige Stiftung, kein Unternehmen

OWASP steht für Open Worldwide Application Security Project (früher: Open Web Application Security Project) und ist eine gemeinnützige Stiftung. Alle Projekte, Dokumente und Werkzeuge entstehen ehrenamtlich und werden kostenlos veröffentlicht – ein wesentlicher Unterschied zu den kommerziellen Herstellern, die in anderen Büchern dieser Reihe vorkommen (etwa PortSwigger bei Burp Suite, Burp-Suite-Handbuch, Kapitel 1.2).

Gut zu wissen

Gegründet im Jahr 2001, veröffentlichte OWASP bereits 2003 die erste Fassung der Top 10. Seither erschienen weitere Fassungen 2004, 2007, 2010, 2013, 2017, 2021 und zuletzt 2025 – die 2025er-Fassung ist damit erst die achte Ausgabe in über zwanzig Jahren.

1.2 Die wichtigsten Projekte im Überblick

Projekt	Zweck
OWASP Top 10	die zehn kritischsten Risikokategorien für Webanwendungen (Kapitel 2-8)
OWASP ASVS (Application Security Verification Standard)	ein detaillierter Kriterienkatalog zur Prüfung, wie gründlich eine Anwendung abgesichert wurde
OWASP Testing Guide	eine ausführliche methodische Anleitung, wie man einzelne Schwachstellenklassen praktisch testet
OWASP Cheat Sheet Series	kompakte, konkrete Umsetzungsempfehlungen zu einzelnen Themen (z. B. Passwort-Speicherung, Session-Management)
OWASP ZAP	ein kostenloses, quelloffenes Werkzeug zur Prüfung von Webanwendungen, vergleichbar mit Burp Suite
OWASP Dependency-Check	ein Werkzeug, das bekannte Schwachstellen in genutzten Programmbibliotheken aufspürt (vertieft in Kapitel 5)

1.3 Warum die Top 10 nicht das ganze Bild sind

WARNUNG – Sicherheitsrisiko

Die Top 10 sind ein **Bewusstseinsdokument**, keine vollständige Prüfvorschrift. Eine Anwendung, die alle zehn Kategorien fehlerfrei abdeckt, kann trotzdem verwundbar sein – durch eine Schwachstelle, die in keine der zehn Kategorien eindeutig fällt, oder durch ein Zusammenspiel mehrerer kleinerer Mängel. Für eine erschöpfende Prüfung dienen ASVS und Testing Guide als Ergänzung, nicht die Top 10 allein.

Beispiel 1.1: Wie die Dokumente zusammenwirken (Konzept)

```
Top 10        -> Welche Risikokategorien sind am wichtigsten? (Einstieg, Priorisierung)
Testing Guide -> Wie teste ich eine konkrete Kategorie praktisch?
ASVS          -> Wie vollständig und wie streng wurde tatsächlich geprüft?
Cheat Sheets  -> Wie setze ich eine konkrete Gegenmaßnahme korrekt um?
```

Zusammenfassung

Zusammenfassung

- OWASP ist eine gemeinnützige Stiftung, deren Projekte kostenlos und ehrenamtlich entstehen.
- Die Top 10 sind nur eines von mehreren OWASP-Projekten, neben ASVS, Testing Guide, Cheat Sheet Series, ZAP und Dependency-Check.
- Seit 2003 erschienen acht Fassungen der Top 10, zuletzt 2025.
- Die Top 10 sind ein Bewusstseinsdokument zur Priorisierung, keine abschließende Prüfvorschrift.

Kapitel 2: Die OWASP Top 10:2025 im Überblick

Lernziele

- Du kennst alle zehn Kategorien der aktuellen Fassung in der richtigen Reihenfolge.
- Du kannst die wichtigsten Verschiebungen gegenüber der Fassung von 2021 benennen.
- Du verstehst, wie OWASP die Reihenfolge der Kategorien ermittelt.

2.1 Die zehn Kategorien

Rang	Kategorie
A01:2025	Broken Access Control
A02:2025	Security Misconfiguration
A03:2025	Software Supply Chain Failures
A04:2025	Cryptographic Failures
A05:2025	Injection
A06:2025	Insecure Design
A07:2025	Authentication Failures
A08:2025	Software and Data Integrity Failures
A09:2025	Logging and Alerting Failures
A10:2025	Mishandling of Exceptional Conditions

2.2 Wie die Reihenfolge entsteht

OWASP ermittelt die Reihenfolge aus zwei Quellen: automatisiert ausgewerteten Daten aus hunderttausenden geprüften Anwendungen (Häufigkeit und Schweregrad bekannter Schwachstellenklassen) sowie einer Befragung der Sicherheitsgemeinschaft für Kategorien, zu denen noch keine ausreichende Datenbasis vorliegt.

Gut zu wissen

Acht der zehn Kategorien der 2025er-Fassung stammen aus der Datenauswertung, zwei (darunter die neue Kategorie Software Supply Chain Failures) wurden ergänzend aus der Community-Befragung aufgenommen, weil die Datenlage zu diesen, vergleichsweise neueren Themen noch nicht ausreichte.

2.3 Was sich seit 2021 verändert hat

Veränderung	Erläuterung
Broken Access Control bleibt Rang 1	unverändert die häufigste Kategorie, nimmt jetzt aber auch Server-Side Request Forgery (SSRF) auf, das 2021 noch eine eigene Kategorie war
Security Misconfiguration springt von Rang 5 auf Rang 2	Fehlkonfigurationen in Cloud-Diensten, Containern und Proxys häufen sich in den Daten deutlich
Software Supply Chain Failures ist neu auf Rang 3	erweitert die 2021er-Kategorie „Vulnerable and Outdated Components" um die gesamte Lieferkette (vertieft in Kapitel 5)
Mishandling of Exceptional Conditions ist komplett neu	Fehlerbehandlung, die selbst zur Schwachstelle wird, war 2021 noch in keiner eigenen Kategorie erfasst
Identification and Authentication Failures heißt jetzt Authentication Failures	der Fokus verengt sich auf die Stellen, an denen tatsächlich Vorfälle entstehen

Beispiel 2.1: Die wichtigsten Verschiebungen auf einen Blick

```
2021 Rang 1 Broken Access Control        -> 2025 Rang 1 (unverändert, jetzt inkl. SSRF)
2021 Rang 2 Cryptographic Failures       -> 2025 Rang 4
2021 Rang 3 Injection                    -> 2025 Rang 5
2021 Rang 5 Security Misconfiguration    -> 2025 Rang 2
2021 Rang 6 Vulnerable/Outdated Comp.    -> 2025 Rang 3 (erweitert: Software Supply Chain)
2021 Rang 10 SSRF (eigene Kategorie)     -> 2025 aufgegangen in A01
```

TIPP – Profi-Trick

Arbeitest du noch mit älteren Unterlagen oder einer Zertifizierung, die sich auf die 2021er-Fassung bezieht (viele Lehrpläne, etwa CEH, wurden zum Zeitpunkt der Erstellung noch nicht vollständig umgestellt), hilft diese Tabelle, zwischen beiden Fassungen zu übersetzen.

Zusammenfassung

Zusammenfassung

- Die zehn Kategorien der Top 10:2025 reichen von Broken Access Control (Rang 1) bis Mishandling of Exceptional Conditions (Rang 10, neu).
- Acht Kategorien stammen aus automatisiert ausgewerteten Prüfdaten, zwei aus einer ergänzenden Community-Befragung.
- Security Misconfiguration und Software Supply Chain Failures sind die auffälligsten Aufsteiger gegenüber 2021.
- SSRF ist keine eigene Kategorie mehr, sondern Teil von Broken Access Control.

Kapitel 3: A01:2025 – Broken Access Control

Lernziele

- Du erkennst die häufigsten Formen von Broken Access Control.
- Du verstehst, warum SSRF jetzt Teil dieser Kategorie ist statt einer eigenen.
- Du kennst die zentralen Gegenmaßnahmen.

3.1 Was gehört zu dieser Kategorie?

Broken Access Control bezeichnet jede Situation, in der eine Anwendung nicht korrekt durchsetzt, wer auf welche Daten oder Funktionen zugreifen darf. Mit 40 zugeordneten CWE-Einträgen (Common Weakness Enumeration, ein standardisierter Katalog von Schwachstellenarten) ist dies die Kategorie mit dem größten Umfang in der gesamten Top 10:2025.

Typische Ausprägung	Beispiel
Insecure Direct Object Reference (IDOR)	eine URL wie `/rechnung?id=1024` lässt sich einfach auf `id=1025` ändern und zeigt fremde Daten
Fehlende Funktionsprüfung	ein regulärer Nutzer kann eine Admin-Funktion aufrufen, deren Link nur im Admin-Menü versteckt war
Path Traversal	`../../etc/passwd` in einem Dateipfad-Parameter
Server-Side Request Forgery (SSRF), jetzt Teil von A01	der Server wird dazu gebracht, eine Anfrage an eine vom Angreifer gewählte, eigentlich nicht vorgesehene Adresse zu senden

3.2 Warum SSRF jetzt hierher gehört

Bei SSRF trickst eine eingegebene URL die Anwendung aus, eine serverseitige Anfrage an ein internes System zu senden, auf das von außen eigentlich kein Zugriff bestehen sollte.

Beispiel 3.1: SSRF als Zugriffskontroll-Problem (Konzept)

```
Eingabefeld "Bild-URL": http://169.254.169.254/latest/meta-data/
-> Statt eines externen Bildes fragt der Server eine interne,
   nur ihm selbst zugängliche Metadaten-Adresse ab.
```

OWASP ordnet SSRF deshalb 2025 nicht mehr als eigene Angriffstechnik ein, sondern als das, was es im Kern ist: eine fehlende Prüfung, **welche** Adressen eine serverseitige Anfrage überhaupt erreichen darf – strukturell also dieselbe Fehlerklasse wie ein IDOR, nur auf Netzwerkebene statt auf Datenebene.

3.3 Mit Burp Suite nachvollziehen

Beispiel 3.2: IDOR im Repeater testen (vergleiche Burp-Suite-Handbuch, Kapitel 5)

```
GET /rechnung?id=1024 HTTP/1.1
Cookie: PHPSESSID=eigene-session
-> Parameterwert im Repeater auf id=1025 ändern und Antwort prüfen
```

ILLEGAL – niemals ohne Auftrag

Teste Zugriffskontroll-Fehler ausschließlich in einer eigenen Laborumgebung wie DVWA oder im Rahmen eines autorisierten Auftrags. Das Abändern einer ID, um fremde Daten einzusehen, ist gegen ein reales System unabhängig vom Ausgang unautorisiert.

3.4 Gegenmaßnahmen

Maßnahme	Wirkung
Zugriffsprüfung serverseitig, nicht nur im Frontend	verhindert, dass eine versteckte Funktion dennoch erreichbar bleibt
Standardmäßig verweigern (Deny by Default)	jede Anfrage muss sich die Berechtigung aktiv verdienen
Allowlist für SSRF-anfällige Funktionen	nur ausdrücklich erlaubte Zieladressen für serverseitige Anfragen zulassen
Zentrale Zugriffskontroll-Logik statt verstreuter Einzelprüfungen	reduziert das Risiko, eine Prüfung an einer Stelle schlicht zu vergessen

Zusammenfassung

Zusammenfassung

- Broken Access Control ist mit 40 CWE-Einträgen die umfangreichste Kategorie der Top 10:2025.
- IDOR, fehlende Funktionsprüfung, Path Traversal und seit 2025 auch SSRF gehören alle hierher.
- SSRF zählt dazu, weil es im Kern eine fehlende Prüfung ist, welche Ziele eine serverseitige Anfrage erreichen darf.
- Serverseitige, standardmäßig verweigernde Prüfungen sind die wirksamste Gegenmaßnahme.

Kapitel 4: A02:2025 – Security Misconfiguration

Lernziele

- Du verstehst, warum diese Kategorie von Rang 5 auf Rang 2 gestiegen ist.
- Du erkennst typische Fehlkonfigurationen in modernen Umgebungen.
- Du kennst die zentralen Gegenmaßnahmen.

4.1 Warum diese Kategorie so stark gestiegen ist

Security Misconfiguration bezeichnet jede Sicherheitslücke, die nicht aus fehlerhaftem Code, sondern aus einer **falschen oder unvollständigen Konfiguration** entsteht. Die Daten zur Top 10:2025 zeigen Fehlkonfigurationen in praktisch jeder geprüften Anwendung – ein deutlich höherer Anteil als 2021, vor allem wegen der Verbreitung von Cloud-Diensten, Containern und automatisiert bereitgestellter Infrastruktur, in der jede einzelne Komponente eine eigene, leicht übersehbare Konfigurationsfläche mitbringt.

Typische Ausprägung	Beispiel
Standard-Zugangsdaten unverändert	ein Admin-Interface bleibt mit `admin/admin` erreichbar
Unnötig offene Cloud-Speicher	ein S3-Bucket oder Blob-Storage ohne Zugriffsbeschränkung
Überflüssige Dienste und Ports aktiv	ein Entwicklungs-Endpunkt bleibt versehentlich in der Produktivumgebung erreichbar
Fehlende Sicherheits-Header	fehlende `Content-Security-Policy` oder `X-Frame-Options`
Ausführliche Fehlermeldungen in Produktion	Stack-Traces verraten interne Pfade oder Versionsnummern

4.2 Mit Werkzeugen aus dieser Reihe aufspüren

Beispiel 4.1: Offene Ports und Dienste erkennen (vergleiche Nmap-Handbuch, Kapitel 4-5)

```
$ nmap -sV -p- 192.168.56.102
```

Beispiel 4.2: Fehlende Sicherheits-Header in Burp Suite prüfen

```
Proxy -> HTTP History -> Antwort-Header einer Seite ansehen
-> Fehlt "Content-Security-Policy"? Fehlt "X-Frame-Options"?
```

Gut zu wissen

Ein auffällig hoher Anteil der in dieser Kategorie gefundenen Probleme lässt sich bereits mit passiver Beobachtung entdecken – ein aktiver Exploit-Versuch ist oft gar nicht nötig, um eine Fehlkonfiguration zu erkennen (vergleiche die passive Scan-Variante in Burp Suite, Burp-Suite-Handbuch, Kapitel 8.3).

4.3 Gegenmaßnahmen

Maßnahme	Wirkung
Gehärtete Standardkonfiguration für jede neue Umgebung	reduziert die Zahl der Dinge, die einzeln korrigiert werden müssen
Automatisierte Konfigurationsprüfung (Infrastructure as Code mit Linting)	findet Abweichungen vor der Bereitstellung statt danach
Minimalprinzip: nur aktivieren, was tatsächlich gebraucht wird	weniger aktive Komponenten bedeuten weniger mögliche Fehlkonfigurationen
Regelmäßige, wiederholte Konfigurationsaudits	Fehlkonfigurationen entstehen oft erst durch spätere Änderungen, nicht beim Ersteinrichten

TIPP – Profi-Trick

Ein wiederkehrender Konfigurationsaudit lässt sich gut mit einem einfachen Nmap-Skript automatisieren (Nmap-Handbuch, Kapitel 9), das regelmäßig prüft, ob sich seit dem letzten Lauf neue, unerwartete offene Ports oder Dienste eingeschlichen haben.

Zusammenfassung

Zusammenfassung

- Security Misconfiguration stieg von Rang 5 (2021) auf Rang 2 (2025), vor allem wegen Cloud- und Container-Umgebungen.
- Typische Formen reichen von unveränderten Standard-Zugangsdaten bis zu fehlenden Sicherheits-Headern.
- Viele dieser Probleme lassen sich bereits durch passive Beobachtung finden, ohne aktiven Angriff.
- Gehärtete Standardkonfigurationen und wiederkehrende Audits sind die wirksamsten Gegenmaßnahmen.

Kapitel 5: A03:2025 – Software Supply Chain Failures

Lernziele

- Du verstehst, wie sich diese Kategorie aus der 2021er-Kategorie „Vulnerable and Outdated Components" entwickelt hat.
- Du kannst mit OWASP Dependency-Check verwundbare Programmbibliotheken aufspüren.
- Du kennst die zentralen Gegenmaßnahmen.

5.1 Mehr als nur veraltete Bibliotheken

2021 hieß diese Kategorie noch „Vulnerable and Outdated Components" und beschränkte sich im Wesentlichen auf eine Frage: Nutzt die Anwendung eine Programmbibliothek mit einer bekannten, öffentlich dokumentierten Schwachstelle? Die 2025er-Fassung erweitert den Blick auf die **gesamte Lieferkette**: nicht nur die Bibliothek selbst, sondern auch, woher sie stammt, wie sie gebaut wurde und wer sie verändern könnte, bevor sie in der eigenen Anwendung landet.

Typische Ausprägung	Beispiel
Bekannte Schwachstelle in einer genutzten Bibliothek	eine veraltete Version einer Webframework-Komponente mit öffentlich dokumentiertem CVE
Kompromittierte Build-Pipeline	ein manipuliertes Paket wird während des automatisierten Build-Prozesses eingeschleust
Fehlende Integritätsprüfung heruntergeladener Pakete	ein Paket wird ohne Prüfsummenabgleich installiert
Typosquatting	ein Paket mit absichtlich ähnlichem Namen wie eine populäre, echte Bibliothek

5.2 Mit OWASP Dependency-Check aufspüren

Beispiel 5.1: Ein Projektverzeichnis auf bekannte Schwachstellen prüfen

```
$ dependency-check --project "Meine Anwendung" --scan ./projektverzeichnis
```

Das Werkzeug gleicht die im Projekt genutzten Bibliotheken mit öffentlichen Schwachstellen-Datenbanken ab und erzeugt einen Bericht mit allen gefundenen Treffern, jeweils mit Schweregrad und betroffener Version.

Gut zu wissen

Dieses Prinzip – eine bekannte, verwundbare Softwareversion anhand ihres Banners oder ihrer Metadaten erkennen – kennst du bereits aus dem Nmap-Handbuch (Kapitel 5.1) und dem Ethical-Hacking-Handbuch (Kapitel 5.3, vsftpd 2.3.4). OWASP Dependency-Check wendet dieselbe Grundidee konsequent auf den gesamten Satz an Programmbibliotheken eines Projekts an, statt nur auf einen einzelnen Netzwerkdienst.

5.3 Gegenmaßnahmen

Maßnahme	Wirkung
Automatisierte Abhängigkeitsprüfung in der Build-Pipeline	findet bekannte Schwachstellen vor der Veröffentlichung
Software Bill of Materials (SBOM) führen	schafft Überblick, welche Komponenten überhaupt im Einsatz sind
Nur aus vertrauenswürdigen, offiziellen Quellen beziehen	reduziert das Risiko manipulierter oder nachgeahmter Pakete
Versionen gezielt und zeitnah aktualisieren	schließt bekannt gewordene Lücken, statt sie dauerhaft offen zu lassen

WARNUNG – Sicherheitsrisiko

Eine veraltete Version allein beweist noch keine tatsächliche Verwundbarkeit der konkreten Anwendung – entscheidend ist, ob die betroffene Funktion der Bibliothek überhaupt genutzt wird. Prüfe gefundene Treffer deshalb inhaltlich, statt automatisch jede gemeldete Version pauschal als kritisch zu behandeln.

Zusammenfassung

Zusammenfassung

- Software Supply Chain Failures erweitert die 2021er-Kategorie „Vulnerable and Outdated Components" um die gesamte Herkunfts- und Baukette einer Komponente.
- OWASP Dependency-Check gleicht genutzte Bibliotheken automatisiert mit bekannten Schwachstellen ab.
- Eine Software Bill of Materials schafft Überblick über tatsächlich genutzte Komponenten.
- Ein gemeldeter Treffer sollte inhaltlich geprüft werden, bevor er als kritisch eingestuft wird.

Kapitel 6: A04:2025 – Cryptographic Failures

Lernziele

- Du erkennst die häufigsten kryptografischen Fehler in Webanwendungen.
- Du kannst den Zusammenhang zu den Grundlagen aus dem John-the-Ripper-Handbuch herstellen.
- Du kennst die zentralen Gegenmaßnahmen.

6.1 Von Rang 2 (2021) auf Rang 4 (2025)

Cryptographic Failures bezeichnet jede Schwäche beim Schutz von Daten durch Verschlüsselung oder Hashing – nicht der Umstand, dass Kryptografie fehlt, sondern dass sie **falsch oder unzureichend** eingesetzt wird. Der Rückgang von Rang 2 auf Rang 4 bedeutet nicht, dass das Thema unwichtiger geworden wäre, sondern dass andere Kategorien (Security Misconfiguration, Software Supply Chain Failures) in den aktuellen Daten noch häufiger auftreten.

Typische Ausprägung	Beispiel
Unverschlüsselte Übertragung sensibler Daten	Anmeldedaten über HTTP statt HTTPS (Wireshark-Handbuch, Kapitel 7.2)
Schwache oder veraltete Hash-Verfahren für Passwörter	unsalted MD5 statt bcrypt (John-the-Ripper-Handbuch, Kapitel 2.2)
Fest einprogrammierte Schlüssel oder Zugangsdaten	ein API-Schlüssel direkt im Quellcode statt in einer gesicherten Konfiguration
Unzureichend zufällige Zufallswerte	vorhersagbare Session-IDs oder Token

6.2 Querverweise zu anderen Büchern dieser Reihe

Beispiel 6.1: Klartext-Risiko sichtbar machen (vergleiche Wireshark-Handbuch, Kapitel 7)

```
Display-Filter in Wireshark: http.authorization
-> Zeigt HTTP-Basic-Auth-Header, die ohne TLS im Klartext übertragen werden
```

Beispiel 6.2: Schwache Passwort-Hashes prüfen (vergleiche John-the-Ripper-Handbuch, Kapitel 2 und Hashcat-Handbuch, Kapitel 2)

```
$ hashcat -b -m 0     # unsalted MD5: sehr schnell zu prüfen
$ hashcat -b -m 3200   # bcrypt: absichtlich sehr viel langsamer
```

Der Geschwindigkeitsunterschied zwischen beiden Benchmarks aus dem Hashcat-Handbuch ist der direkteste, praktisch nachvollziehbare Beleg dafür, warum diese Kategorie überhaupt existiert: Ein schwaches Verfahren macht aus einem gestohlenen Hash fast sofort ein Klartext-Passwort, ein starkes Verfahren macht denselben Diebstahl erheblich weniger wertvoll.

6.3 Gegenmaßnahmen

Maßnahme	Wirkung
TLS für jede Übertragung sensibler Daten, ohne Ausnahme	verhindert Mitlesen auf dem Transportweg
Moderne, absichtlich langsame Hash-Verfahren (bcrypt, scrypt, Argon2)	verlangsamt Cracking-Versuche massiv (John-the-Ripper-Handbuch, Kapitel 2.2)
Schlüssel und Zugangsdaten in einer dedizierten Verwaltung statt im Code	verhindert versehentliches Offenlegen beim Quellcode-Zugriff
Kryptografisch sichere Zufallszahlengeneratoren für Token und Session-IDs	verhindert Vorhersagbarkeit

TIPP – Profi-Trick

Die Grundlagen zu Hashing und Salting aus dieser Kategorie sind im John-the-Ripper-Handbuch (Kapitel 2) bereits ausführlich erklärt und werden hier bewusst nicht wiederholt, nur in ihren Konsequenzen für diese Top-10-Kategorie eingeordnet.

Zusammenfassung

Zusammenfassung

- Cryptographic Failures betrifft falsch oder unzureichend eingesetzte Kryptografie, nicht deren vollständiges Fehlen.
- Unverschlüsselte Übertragung, schwache Hash-Verfahren und fest einprogrammierte Schlüssel sind die häufigsten Ausprägungen.
- Wireshark und Hashcat aus dieser Reihe machen die praktischen Folgen schwacher Kryptografie direkt sichtbar.
- TLS, moderne Hash-Verfahren und dedizierte Schlüsselverwaltung sind die zentralen Gegenmaßnahmen.

Kapitel 7: A05:2025 – Injection

Lernziele

- Du kennst die wichtigsten Injection-Arten und ihr gemeinsames Grundprinzip.
- Du kannst den Zusammenhang zu den bereits bekannten SQL-Injection-Beispielen dieser Reihe herstellen.
- Du kennst die zentralen Gegenmaßnahmen.

7.1 Von Rang 3 (2021) auf Rang 5 (2025)

Injection bezeichnet jede Schwachstelle, bei der nicht vertrauenswürdige Eingaben als Teil eines Befehls oder einer Abfrage interpretiert werden, statt als reine Daten behandelt zu werden. Trotz des Rückgangs von Rang 3 auf Rang 5 bleibt Injection eine der am längsten bekannten und am besten untersuchten Schwachstellenklassen überhaupt.

Injection-Art	Betroffener Kontext
SQL-Injection	Datenbankabfragen
Command Injection	Betriebssystembefehle (vergleiche hping3-Handbuch, Kapitel 6.1, zu `shell=True`)
Cross-Site Scripting (XSS)	im Browser ausgeführtes JavaScript
LDAP-/NoSQL-Injection	Verzeichnisdienste und dokumentenorientierte Datenbanken

7.2 Dasselbe Grundprinzip, verschiedene Kontexte

Beispiel 7.1: Dasselbe Grundmuster in zwei Kontexten (Konzept)

```
SQL-Injection:     id=1' UNION SELECT user, password FROM users -- -
Command Injection: dateiname=x; cat /etc/passwd
```

In beiden Fällen vertraut die Anwendung darauf, dass eine Eingabe nur Daten enthält, und verarbeitet sie stattdessen als Teil eines ausführbaren Befehls. Das Ethical-Hacking-Handbuch (Kapitel 6.1) zeigt dieses Muster bereits praktisch an DVWA, das hping3-Handbuch (Kapitel 6.1) am Beispiel eines unsicheren `subprocess.run(..., shell=True)`-Aufrufs in Python.

Beispiel 7.2: SQL-Injection systematisch im Repeater testen (vergleiche Burp-Suite-Handbuch, Kapitel 5)

```
GET /vulnerabilities/sqli/?id=1' HTTP/1.1
-> Antwort auf eine SQL-Fehlermeldung hin prüfen
```

7.3 Gegenmaßnahmen

Maßnahme	Wirkung
Parametrisierte Abfragen (Prepared Statements)	trennt Daten strikt vom Befehl, unabhängig vom Eingabeinhalt
Allowlist-Validierung von Eingaben	erlaubt nur ausdrücklich zulässige Zeichen oder Werte
Ausgabe-Kodierung je nach Kontext (HTML, JavaScript, SQL)	verhindert, dass eine Eingabe im jeweiligen Zielkontext als Code interpretiert wird
Geringstmögliche Rechte für Datenbank- und Systemzugriffe	begrenzt den Schaden, falls eine Injection trotzdem gelingt

Gut zu wissen

Parametrisierte Abfragen sind die wirksamste Einzelmaßnahme gegen SQL-Injection, weil sie das Problem strukturell lösen, statt nur einzelne gefährliche Zeichen herauszufiltern – dasselbe Allowlist-Prinzip, das im Vibe-Coding-Handbuch dieser Reihe (Kapitel 6.4) bereits am Beispiel einer Projektnamen-Prüfung erklärt wird.

Zusammenfassung

Zusammenfassung

- Injection entsteht, wenn nicht vertrauenswürdige Eingaben als Teil eines Befehls statt als reine Daten verarbeitet werden.
- SQL-Injection, Command Injection, XSS und LDAP-/NoSQL-Injection teilen dasselbe Grundprinzip in unterschiedlichen Kontexten.
- Parametrisierte Abfragen lösen das Problem strukturell und sind die wirksamste Einzelmaßnahme.
- Allowlist-Validierung und kontextgerechte Ausgabe-Kodierung ergänzen den Schutz zusätzlich.

Kapitel 8: A06–A10:2025 – die übrigen fünf Kategorien

Lernziele

- Du kennst die Kernidee jeder der fünf verbleibenden Kategorien.
- Du kannst Insecure Design von den konkreteren, technischen Kategorien abgrenzen.
- Du verstehst, warum Mishandling of Exceptional Conditions eine komplett neue Kategorie ist.

8.1 A06:2025 – Insecure Design

Insecure Design betrifft Schwächen, die bereits in der Planung einer Anwendung angelegt sind, bevor überhaupt eine Zeile Code geschrieben wurde – etwa ein Bestellprozess, der grundsätzlich keine Mengenbegrenzung vorsieht, oder ein Passwort-Reset-Verfahren, das strukturell erraten werden kann.

Gut zu wissen

Insecure Design unterscheidet sich von den übrigen, eher technischen Kategorien dadurch, dass keine noch so sorgfältige Implementierung das Problem beheben kann – die Lücke steckt im Konzept selbst und muss dort behoben werden, nicht im Code.

8.2 A07:2025 – Authentication Failures

Schwächen bei der Identitätsprüfung: fehlende Mehr-Faktor-Authentifizierung für sensible Funktionen, unzureichender Schutz gegen automatisiertes Durchprobieren von Zugangsdaten, oder Sitzungs-Token, die nach der Abmeldung weiterhin gültig bleiben.

Beispiel 8.1: Login-Schutz testen (vergleiche Burp-Suite-Handbuch, Kapitel 6, und Ethical-Hacking-Handbuch, Kapitel 6.4)

```
Intruder mit Angriffstyp "Sniper" gegen das Passwortfeld
-> Prüfen, ob nach mehreren Fehlversuchen eine Sperre einsetzt
```

8.3 A08:2025 – Software and Data Integrity Failures

Schwächen, die entstehen, wenn Software oder Daten ohne ausreichende Prüfung ihrer Unversehrtheit verarbeitet werden – etwa ein automatisches Update, das ohne Signaturprüfung eingespielt wird, oder eine Anwendung, die serialisierte Daten aus nicht vertrauenswürdiger Quelle unverändert wieder einliest.

Gut zu wissen

Diese Kategorie überschneidet sich teilweise mit A03 (Software Supply Chain Failures, Kapitel 5): A03 betrifft, **woher** eine Komponente stammt, A08 betrifft, **ob ihre Unversehrtheit** beim Einsatz tatsächlich geprüft wird.

8.4 A09:2025 – Logging and Alerting Failures

Fehlende oder unzureichende Protokollierung sicherheitsrelevanter Ereignisse, sodass ein Angriff im Nachhinein nicht nachvollzogen werden kann, oder Protokolle, die zwar geschrieben, aber nie ausgewertet werden, sodass ein laufender Angriff unbemerkt bleibt.

Beispiel 8.2: Wie sich das praktisch zeigt (Konzept)

```
Fehlgeschlagene Login-Versuche werden protokolliert,
aber niemand und kein System sieht sich das Protokoll jemals an
-> dieselbe Schwäche wie fehlende Protokollierung selbst
```

8.5 A10:2025 – Mishandling of Exceptional Conditions

Diese komplett neue Kategorie erfasst Fehler, die entstehen, wenn eine Anwendung auf unerwartete Zustände falsch reagiert – ein Fehlerpfad, der versehentlich mehr Rechte gewährt als der Normalfall, oder eine Ausnahmebehandlung, die interne Details in der Fehlermeldung preisgibt.

> **Gut zu wissen**
>
> Dass diese Kategorie erst 2025 eigenständig aufgenommen wurde, heißt nicht, dass das Problem neu wäre – es war bisher nur über mehrere andere Kategorien verstreut und wird jetzt erstmals gebündelt betrachtet.

Zusammenfassung

> **Zusammenfassung**
>
> - Insecure Design betrifft Schwächen im Konzept, die keine Implementierung beheben kann.
> - Authentication Failures und Software/Data Integrity Failures betreffen Identitätsprüfung beziehungsweise Unversehrtheitsprüfung.
> - Logging and Alerting Failures umfasst sowohl fehlende Protokollierung als auch ungenutzte, vorhandene Protokolle.
> - Mishandling of Exceptional Conditions ist komplett neu und bündelt Fehler im Umgang mit unerwarteten Zuständen.

Kapitel 9: Von der Kategorie zum Test

Lernziele

- Du nutzt den OWASP Testing Guide, um eine Kategorie methodisch statt zufällig zu prüfen.
- Du verstehst den Zweck des ASVS als Prüfkatalog.
- Du findest in der Cheat Sheet Series konkrete Umsetzungshilfe zu einer Gegenmaßnahme.

9.1 Vom Risikobewusstsein zum methodischen Test

Die Top 10 sagen, **welche** Risiken am wichtigsten sind (Kapitel 2-8). Sie sagen nicht, **wie** man eine konkrete Anwendung systematisch darauf prüft. Genau diese Lücke schließt der **OWASP Testing Guide**.

Beispiel 9.1: Testing-Guide-Struktur für Access Control (Konzept)

```
1. Rollen und Berechtigungsstufen der Anwendung identifizieren
2. Für jede Funktion prüfen: welche Rolle sollte Zugriff haben?
3. Mit einem Konto niedrigerer Berechtigung gezielt höher privilegierte
   Funktionen aufrufen (vergleiche Kapitel 3.3, IDOR-Test im Repeater)
4. Ergebnis dokumentieren: Zugriff korrekt verweigert oder nicht?
```

9.2 ASVS: wie gründlich wurde geprüft?

Der **Application Security Verification Standard** gliedert Sicherheitsanforderungen in mehrere Prüftiefen, von grundlegenden bis zu besonders strengen Anforderungen für hochkritische Anwendungen. Statt nur zu fragen „ist die Anwendung sicher?", erlaubt ASVS die präzisere Frage „gegen welche Prüftiefe wurde sie tatsächlich verifiziert?".

Gut zu wissen

ASVS wird regelmäßig aktualisiert; die genaue, zum jeweiligen Zeitpunkt aktuelle Versionsnummer lohnt einen Blick auf owasp.org, bevor du dich in einem Bericht oder Angebot darauf beziehst.

9.3 Die Cheat Sheet Series: konkrete Umsetzung

Wo Top 10 und Testing Guide die Risikolage und die Prüfmethode liefern, liefert die **Cheat Sheet Series** die konkrete, technische Umsetzung einer einzelnen Gegenmaßnahme – zum Beispiel, wie Passwörter korrekt gespeichert werden sollten.

Beispiel 9.2: Typischer Aufbau eines Cheat Sheets (Konzept)

```
Thema: Password Storage
- Welches Hash-Verfahren wird empfohlen? (vergleiche John-the-Ripper-Handbuch, Kapitel 2.2)
- Welcher Kostenfaktor ist angemessen?
- Welche Fallstricke sind bekannt und sollten vermieden werden?
```

9.4 Die drei Dokumente im Zusammenspiel

Frage	Dokument
Welches Risiko ist am wichtigsten?	Top 10
Wie teste ich das konkret?	Testing Guide
Wie gründlich wurde tatsächlich geprüft?	ASVS
Wie setze ich die Gegenmaßnahme richtig um?	Cheat Sheet Series

TIPP – Profi-Trick

Ein vollständiger, dokumentierter Prüfbericht (Ethical-Hacking-Handbuch, Kapitel 8) gewinnt an Aussagekraft, wenn er nicht nur Funde auflistet, sondern jeden Fund einer Top-10-Kategorie zuordnet und auf das passende Cheat Sheet für die Gegenmaßnahme verweist.

Zusammenfassung

Zusammenfassung

- Die Top 10 zeigen, welches Risiko wichtig ist; der Testing Guide zeigt, wie man es methodisch prüft.
- ASVS liefert eine abgestufte Prüftiefe statt einer pauschalen Sicher-oder-nicht-Aussage.
- Die Cheat Sheet Series liefert konkrete, technische Umsetzungshilfe zu einzelnen Gegenmaßnahmen.
- Ein guter Prüfbericht verbindet alle drei Dokumente mit den tatsächlich gefundenen Schwachstellen.

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen

Lernziele

- Du baust das für dieses Buch nötige Labor auf.
- Du arbeitest acht Übungen durch, die jeweils einer Top-10-Kategorie zugeordnet sind.
- Du dokumentierst jeden Fund mit der passenden Kategorie-Bezeichnung.

10.1 Laboraufbau

Dieses Buch nutzt dieselbe DVWA-Laborumgebung wie das Ethical-Hacking- und das Burp-Suite-Handbuch, zusätzlich mit installiertem Nmap, Hashcat und OWASP Dependency-Check.

10.2 Checkliste: sichere Laborumgebung

Checkliste 10.1: Vor jeder Übung

```
[ ] DVWA läuft ausschließlich an 127.0.0.1 (Burp-Suite-Handbuch, Kapitel 3.4)
[ ] Burp Suite mit Scope auf DVWA begrenzt
[ ] Jeder Fund wird mit Kategorie-Bezeichnung (z. B. "A01:2025") dokumentiert
```

10.3 Übungen

Jede Übung folgt demselben Schema: Ziel, Voraussetzungen, Durchführung, erwartetes Ergebnis, Auswertung und Fehlerbehebung.

Übung 1: Einen IDOR finden und als A01 dokumentieren

Ziel: Broken Access Control praktisch nachvollziehen und korrekt zuordnen. **Voraussetzungen:** DVWA, Burp Suite im Repeater-Modus. **Durchführung:** Rufe eine DVWA-Funktion mit einer ID im Parameter auf und ändere den Wert im Repeater (Beispiel 3.2). **Erwartetes Ergebnis:** Zugriff auf Daten, die zu einer anderen ID gehören. **Auswertung:** Dokumentiere den Fund als „A01:2025 – Broken Access Control" mit kurzer Beschreibung. **Fehlerbehebung:** Kein Unterschied in der Antwort? Prüfe, ob der Parameter tatsächlich serverseitig ausgewertet wird.

Übung 2: Fehlende Sicherheits-Header als A02 dokumentieren

Ziel: Security Misconfiguration anhand von Antwort-Headern erkennen. **Voraussetzungen:** DVWA, Burp-Suite-HTTP-History. **Durchführung:** Prüfe die Antwort-Header einer DVWA-Seite auf `Content-Security-Policy` und `X-Frame-Options` (Beispiel 4.2). **Erwartetes Ergebnis:** Mindestens ein empfohlener Header fehlt. **Auswertung:** Dokumentiere den Fund als „A02:2025 – Security Misconfiguration". **Fehlerbehebung:** Alle Header vorhanden? Prüfe stattdessen auf ausführliche Fehlermeldungen bei einer absichtlich fehlerhaften Anfrage.

Übung 3: Eine Abhängigkeit mit OWASP Dependency-Check prüfen

Ziel: A03 praktisch mit dem passenden Werkzeug nachvollziehen. **Voraussetzungen:** ein kleines, eigenes Testprojekt mit mindestens einer bekannten, älteren Bibliotheksversion. **Durchführung:** Führe `dependency-check` gegen das Testprojekt aus (Beispiel 5.1). **Erwartetes Ergebnis:** Ein Bericht mit mindestens einem gefundenen, bekannten Schwachstellen-Eintrag. **Auswertung:** Dokumentiere den Fund als „A03:2025 – Software Supply Chain Failures" und prüfe, ob die betroffene Funktion tatsächlich genutzt wird. **Fehlerbehebung:** Keine Treffer? Nutze testweise eine bewusst ältere, öffentlich dokumentiert verwundbare Bibliotheksversion.

Übung 4: Klartext-Zugangsdaten als A04 dokumentieren

Ziel: Cryptographic Failures über die bereits bekannte Wireshark-Übung neu einordnen. **Voraussetzungen:** Wireshark-Handbuch, Übung 5 (bereits durchgeführt oder wiederholt). **Durchführung:** Wiederhole den HTTP-Basic-Auth-Mitschnitt aus dem Wireshark-Handbuch. **Erwartetes Ergebnis:** Zugangsdaten erscheinen lesbar oder leicht dekodierbar. **Auswertung:** Dokumentiere den Fund diesmal als „A04:2025 – Cryptographic Failures", nicht nur als technischen Klartext-Fund. **Fehlerbehebung:** Siehe Fehlerbehebung der Originalübung im Wireshark-Handbuch.

Übung 5: SQL-Injection als A05 dokumentieren

Ziel: Injection über die bereits bekannte Übung neu einordnen. **Voraussetzungen:** Ethical-Hacking-Handbuch, Kapitel 6.1, oder Burp-Suite-Handbuch, Übung 3. **Durchführung:** Wiederhole den SQL-Injection-Test gegen DVWA. **Erwartetes Ergebnis:** Benutzername-Hash-Paare erscheinen in der Antwort. **Auswertung:** Dokumentiere den Fund als „A05:2025 – Injection". **Fehlerbehebung:** Siehe Fehlerbehebung der Originalübung im jeweiligen Buch.

Übung 6: Eine der fünf übrigen Kategorien selbst zuordnen

Ziel: Die in Kapitel 8 beschriebenen Kategorien auf einen eigenen Fund anwenden. **Voraussetzungen:** DVWA mit mehreren Sicherheitsstufen. **Durchführung:** Wähle eine DVWA-Funktion zum Thema Login-Versuche (Brute-Force-Modul) und prüfe, ob nach mehreren Fehlversuchen eine Sperre einsetzt (Beispiel 8.1). **Erwartetes Ergebnis:** Keine erkennbare Sperre, selbst nach vielen Versuchen. **Auswertung:** Ordne den Fund selbstständig einer der fünf Kategorien aus Kapitel 8 zu und begründe deine Wahl schriftlich. **Fehlerbehebung:** Unsicher bei der Zuordnung? Vergleiche deine Begründung mit der Tabelle in Kapitel 9.4.

Übung 7: Einen vollständigen Testing-Guide-Durchlauf simulieren

Ziel: Den methodischen Ablauf aus Kapitel 9.1 selbst anwenden. **Voraussetzungen:** Ergebnisse aus Übung 1. **Durchführung:** Formuliere für den IDOR-Fund aus Übung 1 die vier Schritte aus Beispiel 9.1 schriftlich aus, mit den tatsächlichen Rollen und Funktionen von DVWA. **Erwartetes Ergebnis:** Eine vollständige, nachvollziehbare Testbeschreibung von der Rollendefinition bis zur Dokumentation. **Auswertung:** Vergleiche deine Beschreibung mit dem tatsächlichen Testablauf aus Übung 1. **Fehlerbehebung:** Fällt dir ein Schritt schwer, lies Kapitel 9.1 noch einmal mit Blick auf genau diesen Schritt.

Übung 8: Einen vollständigen Kategorien-Bericht erstellen

Ziel: Alle Funde dieser Übungsreihe zu einem zusammenhängenden Bericht verbinden. **Voraussetzungen:** Ergebnisse aus Übung 1-6. **Durchführung:** Erstelle eine Tabelle mit Spalten „Kategorie", „Fund", „Schweregrad (eigene Einschätzung)" und „empfohlenes Cheat Sheet" für jeden der sechs Funde. **Erwartetes Ergebnis:** Eine vollständige, nach Top-10-Kategorien geordnete Übersicht. **Auswertung:** Vergleiche deine Tabelle mit dem Berichtsschema aus dem Ethical-Hacking-Handbuch (Kapitel 8.2). **Fehlerbehebung:** Fehlt ein passendes Cheat Sheet? Die OWASP-Projektseite listet die Cheat Sheet Series vollständig und durchsuchbar.

Zusammenfassung

Zusammenfassung

- Dieselbe DVWA-Laborumgebung wie in den anderen Büchern dieser Reihe reicht für alle acht Übungen aus.
- Jede Übung verbindet einen bereits bekannten technischen Fund mit seiner korrekten Top-10:2025-Kategorie.
- Der abschließende Bericht verbindet Funde, Kategorien und passende Cheat Sheets zu einem zusammenhängenden Ergebnis.

Kapitel 11: Häufige Missverständnisse, Spickzettel, Glossar und Ressourcen

Lernziele

- Du kannst verbreitete Fehlannahmen über die OWASP Top 10 richtigstellen.
- Du hast einen kompakten Spickzettel für den Alltag zur Hand.
- Du kennst weiterführende Ressourcen für die Vertiefung.

11.1 Häufige Missverständnisse

Verbreitete Annahme	Richtigstellung
„Top 10 konform" bedeutet „sicher"	Die Top 10 sind ein Bewusstseinsdokument, keine vollständige Prüfvorschrift (Kapitel 1.3)
Die Rangfolge gilt für jede Anwendung gleich	Die Rangfolge ist ein statistischer Durchschnitt über viele geprüfte Anwendungen, nicht eine Vorhersage für die eigene
Ein automatisierter Scan deckt alle zehn Kategorien ab	Kategorien wie Insecure Design (Kapitel 8.1) lassen sich kaum automatisiert erkennen
Die 2021er-Fassung ist noch aktuell	Seit Januar 2026 ist die 2025er-Fassung die gültige, aktuelle Version (Vorwort)
SSRF ist eine eigene Kategorie	SSRF ist seit 2025 Teil von A01, Broken Access Control (Kapitel 3.2)
Ein gefundenes CVE in einer Bibliothek bedeutet automatisch eine echte Schwachstelle	Entscheidend ist, ob die betroffene Funktion tatsächlich genutzt wird (Kapitel 5.3)

11.2 Spickzettel

Beispiel 11.1: Die OWASP Top 10:2025 auf einen Blick

```
A01 Broken Access Control              (inkl. SSRF)
A02 Security Misconfiguration
A03 Software Supply Chain Failures
A04 Cryptographic Failures
A05 Injection
A06 Insecure Design
A07 Authentication Failures
A08 Software and Data Integrity Failures
A09 Logging and Alerting Failures
A10 Mishandling of Exceptional Conditions
```

Beispiel 11.2: Welches Werkzeug dieser Reihe für welche Kategorie

```
A01 -> Burp Suite (Repeater, IDOR-Tests)
A02 -> Nmap (offene Ports/Dienste), Burp Suite (Header)
A03 -> OWASP Dependency-Check
A04 -> Wireshark (Klartext), John the Ripper/Hashcat (Hash-Stärke)
A05 -> Burp Suite (Repeater/Intruder), hping3 (Command Injection, konzeptionell)
A07 -> Burp Suite (Intruder, Brute-Force-Schutz)
```

11.3 Glossar

- **ASVS** – Application Security Verification Standard, ein abgestufter Prüfkatalog für Sicherheitsanforderungen.
- **CWE** – Common Weakness Enumeration, ein standardisierter Katalog von Schwachstellenarten, dem einzelne Top-10-Kategorien zugeordnet werden.
- **Insecure Design** – eine Schwäche, die bereits in der Planung angelegt ist, unabhängig von der späteren Implementierung.
- **OWASP** – Open Worldwide Application Security Project, eine gemeinnützige Stiftung für Anwendungssicherheit.
- **SBOM** – Software Bill of Materials, eine vollständige Liste der in einer Anwendung genutzten Komponenten.
- **SSRF** – Server-Side Request Forgery, serverseitig erzwungene Anfragen an eine vom Angreifer gewählte Adresse.
- **Testing Guide** – die OWASP-Anleitung zur methodischen Prüfung einzelner Schwachstellenklassen.
- **Top 10** – das bekannteste OWASP-Projekt, eine priorisierte Liste der zehn kritischsten Risikokategorien.

11.4 Weiterführende Ressourcen

Ressource	Inhalt
owasp.org/Top10/2025	die offizielle, aktuelle Fassung der Top 10
OWASP Cheat Sheet Series (cheatsheetseries.owasp.org)	konkrete Umsetzungshilfe zu einzelnen Gegenmaßnahmen
OWASP Testing Guide	methodische Anleitung zur praktischen Prüfung jeder Kategorie
Ethical-Hacking-, Burp-Suite-, Nmap-, Wireshark-, John-the-Ripper- und Hashcat-Handbuch (dieselbe Buchreihe)	die Werkzeuge, mit denen sich jede Kategorie praktisch untersuchen lässt
PortSwigger Web Security Academy (Burp-Suite-Handbuch, Kapitel 11.4)	kostenlose Übungslabore zu einzelnen Top-10-Kategorien

> **TIPP – Profi-Trick**
>
> Die offizielle OWASP-Projektseite ist in ständiger Bearbeitung; prüfe bei einem wichtigen Bericht oder Angebot immer die zum jeweiligen Zeitpunkt aktuelle Fassung, statt dich auf den Stand dieses Buches zu verlassen.

Zusammenfassung

Zusammenfassung

- Die häufigsten Missverständnisse betreffen die Vollständigkeit der Top 10 und den Umstieg von der 2021er- auf die 2025er-Fassung.
- Der Spickzettel verbindet jede Top-10-Kategorie mit dem passenden Werkzeug aus dieser Buchreihe.
- Die offizielle OWASP-Projektseite, die Cheat Sheet Series und der Testing Guide sind die wichtigsten Anschlussstellen zum Weiterlernen.

Verzeichnis der Befehlsbeispiele

Beispiel 1.1: Wie die Dokumente zusammenwirken (Konzept) … Seite 8
Beispiel 2.1: Die wichtigsten Verschiebungen auf einen Blick … Seite 10
Beispiel 3.1: SSRF als Zugriffskontroll-Problem (Konzept) … Seite 11
Beispiel 3.2: IDOR im Repeater testen (vergleiche Burp-Suite-Handbuch, Kapitel 5) … Seite 12
Beispiel 4.1: Offene Ports und Dienste erkennen (vergleiche Nmap-Handbuch, Kapitel 4-5) … Seite 13
Beispiel 4.2: Fehlende Sicherheits-Header in Burp Suite prüfen … Seite 13
Beispiel 5.1: Ein Projektverzeichnis auf bekannte Schwachstellen prüfen … Seite 15
Beispiel 6.1: Klartext-Risiko sichtbar machen (vergleiche Wireshark-Handbuch, Kapitel 7) … Seite 17
Beispiel 6.2: Schwache Passwort-Hashes prüfen (vergleiche John-the-Ripper-Handbuch, Kapitel 2 und Hashcat-Handbuch, Kapitel 2) … Seite 17
Beispiel 7.1: Dasselbe Grundmuster in zwei Kontexten (Konzept) … Seite 19
Beispiel 7.2: SQL-Injection systematisch im Repeater testen (vergleiche Burp-Suite-Handbuch, Kapitel 5) … Seite 19
Beispiel 8.1: Login-Schutz testen (vergleiche Burp-Suite-Handbuch, Kapitel 6, und Ethical-Hacking-Handbuch, Kapitel 6.4) … Seite 21
Beispiel 8.2: Wie sich das praktisch zeigt (Konzept) … Seite 22
Beispiel 9.1: Testing-Guide-Struktur für Access Control (Konzept) … Seite 23
Beispiel 9.2: Typischer Aufbau eines Cheat Sheets (Konzept) … Seite 23
Checkliste 10.1: Vor jeder Übung … Seite 25
Beispiel 11.1: Die OWASP Top 10:2025 auf einen Blick … Seite 28
Beispiel 11.2: Welches Werkzeug dieser Reihe für welche Kategorie … Seite 29

Index

ASVS – 23, 29
Authentication Failures – 21
Broken Access Control – 11, 25
Cheat Sheet Series – 23, 27
Cryptographic Failures – 17, 26
CWE – 11, 29
Dependency-Check – 15, 25, 26
IDOR – 11, 12, 25, 26
Injection – 19, 26
Insecure Design – 21, 29
Logging and Alerting Failures – 22
Mishandling of Exceptional Conditions – 22
OWASP – 5, 7, 9, 11, 15, 23, 25, 26
SBOM – 29
Security Misconfiguration – 13, 17, 25
Software and Data Integrity Failures – 21
Software Supply Chain Failures – 17, 26
SSRF – 11, 29
Testing Guide – 23, 29
Top 10 – 5, 7, 11, 13, 23, 28, 29

WEB-SECURITY & OWASP

Wo jeder Fund dieser Reihe hingehört

Dieses Handbuch ordnet ein, was die anderen Bände dieser Reihe praktisch finden: die OWASP Top 10:2025, im Januar 2026 veröffentlicht und seither die gültige Fassung. Es erklärt jede der zehn Kategorien einzeln, zeigt die wichtigsten Verschiebungen gegenüber 2021 und verbindet jeden Fund mit dem passenden Werkzeug aus Burp Suite, Nmap, Wireshark, John the Ripper und Hashcat.

DAS ERWARTET DICH

- Die OWASP Top 10:2025 vollständig erklärt, inklusive aller Verschiebungen seit 2021
- A01 Broken Access Control bis A05 Injection einzeln vertieft, mit Querverweisen zur Reihe
- A06 bis A10 im Überblick, darunter die komplett neue Kategorie Mishandling of Exceptional Conditions
- ASVS, Testing Guide und Cheat Sheet Series im Zusammenspiel
- 8 geführte Übungen gegen DVWA, Spickzettel und Glossar

Für wen?

Leser:innen aller Bände dieser Reihe, die ihre Funde einem anerkannten Raster zuordnen möchten, sowie Studierende und Auszubildende, die sich auf OWASP-bezogene Prüfungsinhalte vorbereiten.

32 Seiten · Stand Oktober 2026 · Ergänzung zur gesamten Buchreihe für IT-Sicherheit

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.